

Vulnerability Assessment Methodology for EVCMS

1 Penetration Testing

1.1 Vulnerabilities and Penetration Testing

1.1.1 SQL Injection (CWE-89)

To assess the susceptibility of EVCMS to SQL injection (SQLi) attacks, we conducted a series of controlled tests targeting user input fields. SQLi can occur when applications do not properly sanitize user inputs, allowing adversaries to manipulate backend SQL queries. This vulnerability can lead to unauthorized access to sensitive data or complete database compromise. Using Burp Intruder, we automated the injection of crafted payloads into available input vectors, primarily targeting login interfaces. The vulnerable web portals will respond with error messages or server delays to the SQLi. The results indicate that most EVCMS do not have exploitable SQLi vectors. Most systems implement secure mechanisms and do not exhibit exposed input fields beyond the login page. Consequently, we observed that EVCMS platforms generally follow good practices that minimize SQLi attack surfaces. These findings suggest that SQLi remains a limited but plausible risk, especially if authentication implementations are misconfigured or outdated.

1.1.2 XML External Entity Injection (XXE) (CWE-611)

To evaluate the presence of XXE vulnerabilities, we performed tests targeting API endpoints using malicious XML requests. XXE attacks exploit misconfigured XML parsers that improperly handle external entity references, allowing attackers to access local or remote files, perform DoS attacks, or execute remote code. Our methodology involved crafting malicious XML payloads that included external entity declarations, then submitting them to EVCMS APIs. We monitored system responses and logged network traffic to identify whether the injected payloads were processed or blocked. These attacks attempt to retrieve sensitive internal data (e.g., /etc/passwd on Linux-based servers). During testing, we observed whether the payloads triggered any outbound network requests or unauthorized file disclosures.

1.1.3 Hard-Coded Credentials (CWE-798)

To identify the presence of hard-coded credentials within EVCMS, we tested both firmware and web applications. Hard-coded credentials refer to static usernames, passwords, or API keys embedded within source code, configuration files, or compiled binaries, making them accessible to an attacker with system or file access. For firmware analysis, we extracted images using Binwalk to examine system configuration files and embedded scripts for plaintext credentials. Any discovery of hard-coded secrets in firmware represents a critical security flaw, as these credentials can often grant privileged access to backend services or administrative interfaces. For the web applications, we inspected client-side JavaScript and HTML code, and examined network traffic to identify if any API keys or tokens were exposed in request headers or payloads. We performed pattern-based scanning for strings such as `password`, `api_key`, and `secret_key`. This methodology allows us to uncover credential exposure risks that could facilitate unauthorized access or lateral movement within the EVCMS ecosystem.

1.1.4 Weak Password Requirements (CWE-521)

Weak Password Requirements represent when systems enforce inadequate password complexity policies, thereby increasing susceptibility to brute-force attacks and credential guessing. In certain cases, systems even permit the continued use of default credentials beyond the initial log-in. The problem is even exacerbated when systems lack rate limiting, allowing brute force attacks. Our assessment focused on identifying minimal password enforcement criteria by analyzing vendor documentation and user manuals and observing behavior during login attempts via web portals. Specific indicators of weak policies included acceptance of short passwords, lack of enforcement for uppercase and lowercase character combinations, omission of special character requirements, and tolerance for dictionary words or sequential numeric patterns. The presence of such weak policies significantly increases the risk of unauthorized access, even more so when combined with predictable usernames.

1.1.5 Use of Default Credentials (CWE-1392)

Default credentials, similar results as hard-coded credentials, allow attackers to gain unauthorized access to an EVCMS, yet without requiring sophisticated attack techniques. Default credentials indicate an EVCMS applies preconfigured, publicly known credentials, e.g., admin/admin, root, or vendor-specific default usernames/passwords. We collect a list of potential default passwords from a common password dictionary and vendor manuals. We automate the testing of these login pairs and monitor for successful authentication signals. We terminate the session beyond this point for ethical considerations.

1.1.6 Server-Side Request Forgery (SSRF) (CWE-918)

To evaluate the presence of SSRF vulnerabilities, we tested whether API endpoints could be manipulated to trigger unauthorized outbound requests from the server. SSRF occurs when an application allows user-supplied URLs to be fetched by the backend, potentially enabling attackers to bypass network boundaries and access internal services. Our testing involved submitting crafted payloads containing external URLs via exposed API endpoints, using Burp Suite. We monitored for successful callbacks and interactions such as DNS lookups or HTTP pings, as an indication of SSRF behavior.

1.1.7 Comma-Separated Values Injection (CSVi) (CWE-1236)

To assess the potential for CSVi, we tested whether user-controlled input was improperly stored in CSV files without adequate sanitization. CSVi vulnerabilities arise when applications export user data directly into CSV format. We detect the CSVi by preparing the list of CSV Injection payloads. If the payloads appear unchanged in the CSV or CSV file execute the formula, we conclude that the EVCMS does not properly sanitize the inputs. This is a rare vulnerability because most EVCMS are designed without storing any CSV data.

1.1.8 Cross-Site Scripting (XSS) (CWE-79)

XSS vulnerabilities occur when web applications fail to adequately sanitize user-supplied input, allowing malicious actors to inject malicious scripts, leading to session hijacking, information theft, and other attacks. To identify XSS vulnerabilities in EVCMS, we injected JavaScript payloads that are designed to trigger simple alerts. Successful execution of the payload, as evidenced by the appearance of a browser alert, indicates that the input was reflected in the response without proper sanitization. We used Burp Suite to automate input fuzzing and inspect server responses for reflected or stored script execution. Any instance where user-supplied scripts were executed was considered a confirmed XSS vulnerability.

1.1.9 Cross-Site Request Forgery (CSRF) (CWE-352)

CSRF vulnerabilities arise when a web application fails to verify that a request was intentionally initiated by an authenticated user. In such cases, attackers can trick logged-in users into unknowingly performing unintended actions, such as modifying configurations or triggering transactions. These payloads were embedded in externally controlled resources (e.g., malicious HTML pages) and executed within an authenticated session to test whether the system would process them without user consent or CSRF token validation. By analyzing the response behavior, we assessed whether critical operations could be triggered without explicit user intent.

1.1.10 Cross-Origin Resource Sharing (CORS) Misconfiguration (CWE-942)

CORS Misconfigurations occur when a web application improperly allows requests from untrusted or arbitrary domains. This exposes sensitive APIs to unauthorized access, potentially enabling data theft, session hijacking, or unauthorized actions initiated from malicious websites. We test for CORS by examining permissive rules in cross-domain policy files. Specifically, we send requests and change the Origin header. A secure application should reject the request. Otherwise, indicates the existence of CORS misconfiguration.

1.1.11 Flash Cross-Domain Policy (FCDP) Misconfiguration (CWE-942)

FCDP misconfigurations occur when attackers are allowed to manipulate Flash-based applications to access unauthorized resources. To detect FCDP misconfigurations, we followed a methodology similar to CORS testing by modifying client-side parameters and sending a modified request to the server. A secure web application will reject the request, and an FCDP misconfigured EVCMS will accept the modified value.

1.1.12 Information Exposure (CWE-200)

Information exposure vulnerabilities occur when sensitive data is unintentionally disclosed to unauthorized users, potentially leading to privacy breaches, user profiling, or system exploitation. The leaked sensitive data might be user email address, EV charging status, API keys, diagnostic information, etc. We scanned for the exposed API keys, email addresses, and system details error messages. For the collected exposed emails, we manually verified whether or not they are the vendor's commercial email.

1.1.13 Forced Browsing (CWE-425)

This involves guessing URL paths to exfiltrate the directory structure of web applications to access hidden resources. This vulnerability allows unauthorized users to bypass navigation controls and access sensitive data or restricted pages directly, simply by entering crafted URLs. To evaluate EVCMS platforms for forced browsing vulnerabilities, we scan the web portal with a list of predictable URL patterns and monitor the page's authentication requirement. Any EVCMS that exposes real-time EV or CS status to unauthenticated users is considered to be vulnerable to Forced browsing. This evaluation was done on EVCMSs that do not publicly advertise this information, and it was discovered through path enumeration.

1.1.14 Missing Authentication (CWE-306)

Missing authentication allows unauthorized users to access functionality that should normally be restricted. We test by removing authentication tokens and replaying requests and monitoring for success. The Burp Active Scanner can also automatically scan api endpoint and web paths to detect missing authentication.

1.1.15 Lack of Rate Limiting (CWE-799)

Lack of rate limiting allows attackers to attempt brute-force attacks or overwhelm the system with requests to perform DoS. We test by automating a cycle of HTTP login requests in a short time. Due to ethical concerns, we limit our requests to 50 times to avoid overloading the tested server.

1.1.16 Cleartext Transmission (CWE-319)

Cleartext transmission of sensitive data, such as passwords or tokens, without encryption, makes this data vulnerable to interception. The attacker could intercept data and perform Man-in-the-Middle (MITM) Attacks. A clear indicator is the use of HTTP, meaning the data is not secured using TLS when HTTPS is used. We also use the Burp Suite to capture HTTP traffic and record cookies, and look for sensitive data in the request and response.

1.1.17 Improper Certificate Validation (CWE-295)

Improper certificate validation occurs when an application fails to correctly verify the authenticity of SSL/TLS certificates during secure communications. This vulnerability exposes systems to MITM attacks, enabling adversaries to intercept, modify, or spoof encrypted traffic. The impact is comparable to Cleartext Transmission, as it undermines the confidentiality and integrity of data being exchanged. Although the system has SSL/TLS authentication, it accepts an attacker's self-signed or malicious certificates. To evaluate EVCMS for ICV, we configured a Burp Suite proxy with a self-signed or forged TLS certificate and intercepted HTTPS requests originating from the client interface or backend communication modules. If the EVCMS accepted the untrusted certificate and established a connection without warning or rejection, it was flagged as vulnerable.